

MANUAL DE INTEGRAÇÃO DO SSO - SEFAZ-MA

Versão: 2.4. | Data: 18/05/2024
Responsável: COTEC

SECRETARIA DA
FAZENDA



Célula de Gestão de Pessoas e Administração
Corpo Técnico para Tecnologia da Informação

Histórico de Alterações

Data	Versão	Responsável	Descrição
01/05/2020	1.0	Luis Roberto	Criação do documento
16/05/2022	2.0	Adriano dos Santos	Atualização do Documento com a nova arquitetura Angular + SpringBoot
18/05/2022	2.1	Adriano dos Santos e Roberto	Correções e complementos
18/08/2022	2.2	Adriano dos Santos	Evoluções do SSO
18/01/2023	2.3	Adriano dos Santos	Versões corretas do SSO
28/05/2024	2.4	Adriano dos Santos	Orientação sobre Client

Sumário

1. INTRODUÇÃO	4
2. CONSIDERAÇÕES INICIAIS.....	4
3. ARQUITETURA DE COMUNICAÇÃO.....	4
3.1 Modelo Conceitual	4
3.2 Modelo Operacional	5
3.3 Padrões Técnicos	5
3.3.1 Padrão de Comunicação SSO	5
3.3.2 Padrão de Comunicação com o Web Service de Consulta de Acessos	6
4. Autenticação SSO	6
4.1 Protocolo Utilizado	6
4.2 Plataformas suportadas.....	6
4.3 Adaptadores Cliente	7
4.4 Parametros do Cliente SpringBoot	8
4.5 Parametros do Cliente Angular	9
4.6 Recepção do Token.....	9
5. SOLICITAÇÃO DE INTEGRAÇÃO.....	13

1. INTRODUÇÃO

Este documento tem como objetivo definir as especificações e critérios técnicos necessários para a utilização do Single Sign On - SSO para uso em sistemas que necessitem integrar com o controle de acesso da SEFAZ-MA.

Importante ressaltar que este documento está voltado principalmente aos sistemas java-web implementados com Angular e SpringBoot que estão definidos na Nova Arquitetura de TI da SEFAZ-MA.

Tanto o SSO quanto a nova arquitetura estão definidos no âmbito do projeto PROFISCO 2 pertencente a Iniciativa Estratégica: 9.5 - Definir a arquitetura e metodologia sw sistemas da SEFAZ.

2. CONSIDERAÇÕES INICIAIS

Anteriormente, toda vez que um usuário precisasse passar de um aplicativo para outro, tinha que inserir suas credenciais, e eventualmente, com um mesmo usuário com logins e senhas diferentes em cada sistema.

O logon único (SSO, single sign-on) é um recurso de autenticação que permite aos usuários terem acesso a vários aplicativos com um único nome de usuário e senha de login sem a necessidade de autenticar a cada novo sistema.

3. ARQUITETURA DE COMUNICAÇÃO

3.1 Modelo Conceitual

A SEFAZ-MA disponibiliza os seguintes serviços:

- a) Autenticação
- b) Autorização

O usuário ao acessar o sistema integrado será redirecionado automaticamente para a tela de login do SSO e após autenticado será redirecionado para o sistema de origem.

O sistema de origem irá consultar os acessos do usuário por meio de Web Service fornecido pela SEFAZ integrado ao Controle de Acesso Centralizado ou, se for necessário, pelo próprio sistema de origem.

Usuários que já estiverem logados no SSO não precisarão realizar o login novamente em qualquer outro sistema integrado.

3.2 Modelo Operacional

O usuário ao acessar um sistema integrado ao SSO, deverá ser verificado se o mesmo já não está autenticado no SSO. Caso já esteja autenticado, o sistema integrado deverá prover autenticação automática com base no token retornado pelo SSO, caso contrário, haverá um redirecionamento para a tela do SSO da SEFAZ onde será realizada a autenticação, como observado na figura 1.

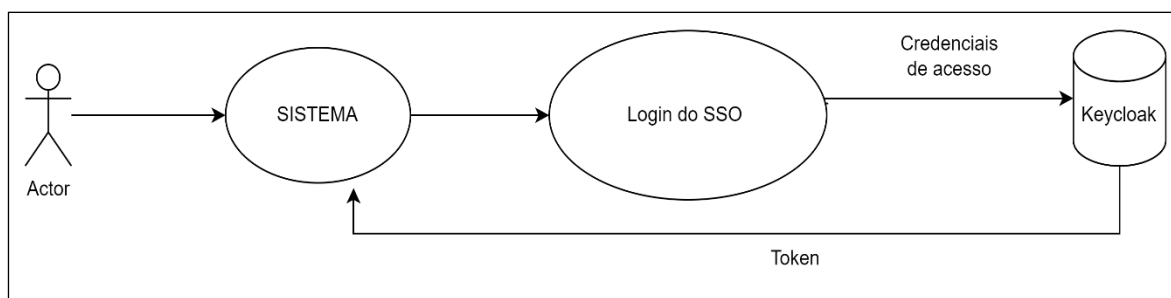


Figura 1 -Diagrama do Login utilizando SSO

O SSO então retornará para o sistema integrado um token com os dados básicos do usuário. Este sistema deverá prover em sua aplicação um arquivo de configuração com a parametrização de redirecionamentos assim como a identificação do sistema integrado.

A consulta dos acessos será feita através de consumo Web Service Rest utilizando o token fornecido pelo SSO e o código de identificação do sistema integrado fornecido pela SEFAZ através da api do Controle de Acesso Centralizado ou, se for necessário, pelo próprio sistema de origem.

3.3 Padrões Técnicos

3.3.1 Padrão de Comunicação SSO

A autenticação do usuário é feito através do uso do SSO da Red Hat Single Sign-On RHSSO 7.5.2 GA que utiliza o software KeyCloack 15.0.6.

A documentação do KeyCloack assim como instruções para instalação e testes pode ser encontrado em <https://www.keycloak.org/documentation>

A documentação do SSO da Red Hat poder ser encontrada em https://access.redhat.com/documentation/en-us/red_hat_single_sign-on/7.2/

O protocolo utilizado será o OpenID Connect.

O Keycloak é um software da Redhat para gerenciamento de identidade e acesso voltado para WebApps e RESTful web services, sendo o responsável pelo produto RH-SSO.

De forma geral, o funcionamento do SSO se dará da seguinte forma: o aplicativo solicita ao servidor Keycloak para autenticar um usuário para eles. Após um login bem-sucedido, o aplicativo receberá um *token de identidade* e um *token de acesso*. O *token de identidade* contém informações sobre o usuário, como nome de usuário, email e outras informações de perfil. O *token de acesso* é assinado digitalmente e contém informações de acesso (como mapeamentos de função do usuário).

Importante ressaltar que a SEFAZ-MA trabalha com o padrão de um Client por sistema ou software, não importando se esse sistema é monolítico ou se possuem front-end e back-end distintos, em que todas as aplicações que compõem este sistema deverão atender a um mesmo Client.

3.3.2 Padrão de Comunicação com o Web Service de Consulta de Acessos

Será utilizado Web Service Rest, realizando um POST em URL definida pela SEFAZ, devendo ser informado os parâmetros “token”, “idusuario” e “sistema” no header da solicitação.

O retorno da consulta será uma lista contendo as aplicações, perfis ou papéis de acesso do usuário para aquele sistema.

4. Autenticação SSO

4.1 Protocolo Utilizado

O protocolo utilizado será o OpenID Connect que é um protocolo de autenticação sendo uma extensão do OAuth 2.0. Embora o OAuth 2.0 seja apenas uma estrutura para a construção de protocolos de autorização e esteja principalmente incompleto, o OIDC é um protocolo completo de autenticação e autorização. O OIDC também faz uso pesado do conjunto de padrões Json Web Token (JWT).

4.2 Plataformas suportadas

Java

- JBoss EAP
- Wildfly
- Fuse
- Tomcat
- Jetty 8
- Servlet Filter
- Spring Boot
- Spring

Security

Frameworks (lado do cliente)

- Angular
- JSF 2.2

Node.js (lado do servidor)

- Node.js

Para Aplicações android :

- AppAuth (generico)
- AeroGear

(generico)Para

Aplicações IOS:

- AppAuth (generico)
- AeroGear (generico)

Este manual irá contemplar com mais detalhes aplicações providas em plataforma Java-web, principalmente aos sistemas implementados com Angular e SpringBoot, no entanto, qualquer plataforma de integração é bem detalhado na documentação oficial.

4.3 Adaptadores Cliente

O Keycloak vem com uma variedade de adaptadores diferentes para aplicativos Java. A seleção do adaptador correto depende da plataforma de destino.

Todos os adaptadores Java compartilham um conjunto de opções de configuração comuns que podem ser configurado por um arquivo JSON simples

chamado de keycloak.json.

Para Springboot, existe um adaptador Keycloak que capitaliza a configuração automática do Spring Boot , tendo apenas que adicionar o inicializador do Keycloak Spring Boot ao projeto.

```
<dependency>
  <groupId>org.keycloak</groupId>
  <artifactId>keycloak-spring-boot-starter</artifactId>
</dependency>
```

Para o Angular, existe um adaptador Keycloak que capitaliza a configuração automática do Spring Boot , tendo apenas que adicionar as bibliotecas do Keycloak ao projeto.

```
// instala bibliotecas

install keycloak-angular@7.1.0 --ou superior
npm install keycloak-js@8.0.2 --ou superior
```

4.4 Parametros do Cliente SpringBoot

Na atual arquitetura que utiliza o sistema implementado no front-end com Angular e no back-end com springboot, toda a comunicação do sistema com o keycloak passa por esta configuração básica e obrigatória que pode ser vista abaixo

```
keycloak.auth-server-url=http://localhost:8180/auth
keycloak.realm=SpringBootKeycloak
keycloak.resource=login-app
keycloak.public-client=true
```

Com isso, temos os seguintes parametros básico necessários para o sistema solicitar a autenticação com o keycloak, realizando o redirecionamento correto para tela do SSO-SEFAZ-MA:

realm

Nome do contexto da integração.

resource

O ID do cliente do aplicativo. Cada aplicativo possui um ID de cliente usado para identificar o aplicativo.

auth-server-url

O URL base do servidor Keycloak. Todas as outras páginas Keycloak e terminais de serviço REST são derivados disso. Geralmente é da forma <https://host:port/auth>.

4.5 Parametros do Cliente Angular

Para conexão do FrontEnd com o SSO, recomenda-se a utilização da biblioteca keycloak javascript, no entanto outras podem ser utilizadas.

Os parametros necessários para integração da aplicação Angular com o SSO são:

```
new Keycloak(  
  {  
    url:URL_SSO,  
    realm :REALM_INTERNO,  
    clientId :CLIENT_IID  
  });
```

- url : URL do SSO (fornecido pela SEFAZ)
- realm : Nome do realm (fornecido pela SEFAZ)
- clientId : Identificador da aplicação cliente

4.6 Recepção do Token

Após o SSO realizar a autenticação e validação, é retornado para a aplicação em uma página de retorno pré-definida o token codificado em JWT. No token estão presentes as informações sobre o usuario e permissões de acesso.

O sistema integrado deverá decodificar o token, extrair as informações devidas e realizar o tratamento necessário sobre as permissões recebidas. Assim, o sistema que deverá implementar as regras de tratamento sobre os perfis informados no token.

Abaixo segue um exemplo de token codificado retornado:

eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJGSjGNsMWVQNURiYazVaYzVNT2p
oUUNtVE5KQ19nRU5WTkhTck80UVpNIn0.eyJleHAiOiJlE2NTI3MjkyOTAsImhhdCI6MTY1MjcyOD
k5MCwiYXV0aF90aW1lIjoxNjUyYzI4OTg4LjQqdGkiOiJlMDNiYWlXOS0xODM4LTQwY2QtOWJ
kZC1jZmY5MmI2Njk0YmliLCJpc3MiOiJodHRwczovL3Nzby1kZXhYtc3NvLWRldi5hcHBzLm9jcC5
zZWZhei5tYS5nb3YuYnIvYXV0aC9yZWZfbWV2YmYXotaW50ZXJub3MiLCJhdWQiOiJhY2Nv
dW50Iiwic3VlIjoiZDVjZDBlYWYtMDVjYi00MmUyLTg3MjctYTYyZDRjMWFjNjk2IiwidHlwIjoiQ
mVhcmVyIiwiaXNjaXVjY21leC1wdWJsaWMtY2xpZW50LWRldiIsIm5vbmlIjoiYTUzNzk5M
TUtYWYzNC00NTM1LTg3N2YtYzVhYzJiN2U0MDIiwic2Vzc2lvdj9zdGF0ZSI6ImZkMmU5Yzdh
LTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImFjciI6IjEiLCJhbGxvd2VkLW9yaWdpbn
MiOlsiaHR0cDovL3B1Y29tZXgt2ViLWFwcC0yLmFwcHMub2NwLnNiZmF6Lm1hLmdvdi5iciIsIm
h0dHBzOi8vcHVjY21leC1kZXhYuc2VmYXoubWEuZ292LmJyIiwiaHR0cDovL2xvY2FsaG9zdDo0Mj
AwIl0sInJlYWxtX2FjY2VzcyI6eyJyb2xlcYI6WyJkZWZhdWx0LXJvbGVzLXNzby1kZXhYIiLCJvZmZ
saW5lX2FjY2VzcyIsInR5cHdXRob3JpemF0aW9uIl19LCJyZXNvdXJjZV9hY2Nlc3MiOnciHVjb2
1leC1wdWJsaWMtY2xpZW50LWRldiI6eyJyb2xlcYI6WyJnZXN0b3ItcHVjY21leCJdfSwiYWNjb3Vu
dCI6eyJyb2xlcYI6WyJtYW5hZ2UuYWNjb3VudCI6Im1hbmFnZS1hY2NvdW50LWxpbnmtzIiwidmllZy
1wcm9maWxlll19fSwic2NvcGUiOiJvcGVuaWQgcHJvZmlsZSBibWVpbCIsInNpZCI6ImZkMmU5Yz
dhLTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImVtYWlsX3ZlcmllmaWVkJjpmYWxzZ
SwibmFtZSI6ImRlc3RlIGdlc3RvcjBwdWNvbWV4Iiwic2Vzc2lvdj9zdGF0ZSI6ImZkMmU5Yzdh
LTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImFjciI6IjEiLCJhbGxvd2VkLW9yaWdpbn
MiOlsiaHR0cDovL3B1Y29tZXgt2ViLWFwcC0yLmFwcHMub2NwLnNiZmF6Lm1hLmdvdi5iciIsIm
h0dHBzOi8vcHVjY21leC1kZXhYuc2VmYXoubWEuZ292LmJyIiwiaHR0cDovL2xvY2FsaG9zdDo0Mj
AwIl0sInJlYWxtX2FjY2VzcyI6eyJyb2xlcYI6WyJkZWZhdWx0LXJvbGVzLXNzby1kZXhYIiLCJvZmZ
saW5lX2FjY2VzcyIsInR5cHdXRob3JpemF0aW9uIl19LCJyZXNvdXJjZV9hY2Nlc3MiOnciHVjb2
1leC1wdWJsaWMtY2xpZW50LWRldiI6eyJyb2xlcYI6WyJnZXN0b3ItcHVjY21leCJdfSwiYWNjb3Vu
dCI6eyJyb2xlcYI6WyJtYW5hZ2UuYWNjb3VudCI6Im1hbmFnZS1hY2NvdW50LWxpbnmtzIiwidmllZy
1wcm9maWxlll19fSwic2NvcGUiOiJvcGVuaWQgcHJvZmlsZSBibWVpbCIsInNpZCI6ImZkMmU5Yz
dhLTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImVtYWlsX3ZlcmllmaWVkJjpmYWxzZ
SwibmFtZSI6ImRlc3RlIGdlc3RvcjBwdWNvbWV4Iiwic2Vzc2lvdj9zdGF0ZSI6ImZkMmU5Yzdh
LTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImFjciI6IjEiLCJhbGxvd2VkLW9yaWdpbn
MiOlsiaHR0cDovL3B1Y29tZXgt2ViLWFwcC0yLmFwcHMub2NwLnNiZmF6Lm1hLmdvdi5iciIsIm
h0dHBzOi8vcHVjY21leC1kZXhYuc2VmYXoubWEuZ292LmJyIiwiaHR0cDovL2xvY2FsaG9zdDo0Mj
AwIl0sInJlYWxtX2FjY2VzcyI6eyJyb2xlcYI6WyJkZWZhdWx0LXJvbGVzLXNzby1kZXhYIiLCJvZmZ
saW5lX2FjY2VzcyIsInR5cHdXRob3JpemF0aW9uIl19LCJyZXNvdXJjZV9hY2Nlc3MiOnciHVjb2
1leC1wdWJsaWMtY2xpZW50LWRldiI6eyJyb2xlcYI6WyJnZXN0b3ItcHVjY21leCJdfSwiYWNjb3Vu
dCI6eyJyb2xlcYI6WyJtYW5hZ2UuYWNjb3VudCI6Im1hbmFnZS1hY2NvdW50LWxpbnmtzIiwidmllZy
1wcm9maWxlll19fSwic2NvcGUiOiJvcGVuaWQgcHJvZmlsZSBibWVpbCIsInNpZCI6ImZkMmU5Yz
dhLTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImVtYWlsX3ZlcmllmaWVkJjpmYWxzZ
SwibmFtZSI6ImRlc3RlIGdlc3RvcjBwdWNvbWV4Iiwic2Vzc2lvdj9zdGF0ZSI6ImZkMmU5Yzdh
LTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImFjciI6IjEiLCJhbGxvd2VkLW9yaWdpbn
MiOlsiaHR0cDovL3B1Y29tZXgt2ViLWFwcC0yLmFwcHMub2NwLnNiZmF6Lm1hLmdvdi5iciIsIm
h0dHBzOi8vcHVjY21leC1kZXhYuc2VmYXoubWEuZ292LmJyIiwiaHR0cDovL2xvY2FsaG9zdDo0Mj
AwIl0sInJlYWxtX2FjY2VzcyI6eyJyb2xlcYI6WyJkZWZhdWx0LXJvbGVzLXNzby1kZXhYIiLCJvZmZ
saW5lX2FjY2VzcyIsInR5cHdXRob3JpemF0aW9uIl19LCJyZXNvdXJjZV9hY2Nlc3MiOnciHVjb2
1leC1wdWJsaWMtY2xpZW50LWRldiI6eyJyb2xlcYI6WyJnZXN0b3ItcHVjY21leCJdfSwiYWNjb3Vu
dCI6eyJyb2xlcYI6WyJtYW5hZ2UuYWNjb3VudCI6Im1hbmFnZS1hY2NvdW50LWxpbnmtzIiwidmllZy
1wcm9maWxlll19fSwic2NvcGUiOiJvcGVuaWQgcHJvZmlsZSBibWVpbCIsInNpZCI6ImZkMmU5Yz
dhLTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImVtYWlsX3ZlcmllmaWVkJjpmYWxzZ
SwibmFtZSI6ImRlc3RlIGdlc3RvcjBwdWNvbWV4Iiwic2Vzc2lvdj9zdGF0ZSI6ImZkMmU5Yzdh
LTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImFjciI6IjEiLCJhbGxvd2VkLW9yaWdpbn
MiOlsiaHR0cDovL3B1Y29tZXgt2ViLWFwcC0yLmFwcHMub2NwLnNiZmF6Lm1hLmdvdi5iciIsIm
h0dHBzOi8vcHVjY21leC1kZXhYuc2VmYXoubWEuZ292LmJyIiwiaHR0cDovL2xvY2FsaG9zdDo0Mj
AwIl0sInJlYWxtX2FjY2VzcyI6eyJyb2xlcYI6WyJkZWZhdWx0LXJvbGVzLXNzby1kZXhYIiLCJvZmZ
saW5lX2FjY2VzcyIsInR5cHdXRob3JpemF0aW9uIl19LCJyZXNvdXJjZV9hY2Nlc3MiOnciHVjb2
1leC1wdWJsaWMtY2xpZW50LWRldiI6eyJyb2xlcYI6WyJnZXN0b3ItcHVjY21leCJdfSwiYWNjb3Vu
dCI6eyJyb2xlcYI6WyJtYW5hZ2UuYWNjb3VudCI6Im1hbmFnZS1hY2NvdW50LWxpbnmtzIiwidmllZy
1wcm9maWxlll19fSwic2NvcGUiOiJvcGVuaWQgcHJvZmlsZSBibWVpbCIsInNpZCI6ImZkMmU5Yz
dhLTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImVtYWlsX3ZlcmllmaWVkJjpmYWxzZ
SwibmFtZSI6ImRlc3RlIGdlc3RvcjBwdWNvbWV4Iiwic2Vzc2lvdj9zdGF0ZSI6ImZkMmU5Yzdh
LTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImFjciI6IjEiLCJhbGxvd2VkLW9yaWdpbn
MiOlsiaHR0cDovL3B1Y29tZXgt2ViLWFwcC0yLmFwcHMub2NwLnNiZmF6Lm1hLmdvdi5iciIsIm
h0dHBzOi8vcHVjY21leC1kZXhYuc2VmYXoubWEuZ292LmJyIiwiaHR0cDovL2xvY2FsaG9zdDo0Mj
AwIl0sInJlYWxtX2FjY2VzcyI6eyJyb2xlcYI6WyJkZWZhdWx0LXJvbGVzLXNzby1kZXhYIiLCJvZmZ
saW5lX2FjY2VzcyIsInR5cHdXRob3JpemF0aW9uIl19LCJyZXNvdXJjZV9hY2Nlc3MiOnciHVjb2
1leC1wdWJsaWMtY2xpZW50LWRldiI6eyJyb2xlcYI6WyJnZXN0b3ItcHVjY21leCJdfSwiYWNjb3Vu
dCI6eyJyb2xlcYI6WyJtYW5hZ2UuYWNjb3VudCI6Im1hbmFnZS1hY2NvdW50LWxpbnmtzIiwidmllZy
1wcm9maWxlll19fSwic2NvcGUiOiJvcGVuaWQgcHJvZmlsZSBibWVpbCIsInNpZCI6ImZkMmU5Yz
dhLTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImVtYWlsX3ZlcmllmaWVkJjpmYWxzZ
SwibmFtZSI6ImRlc3RlIGdlc3RvcjBwdWNvbWV4Iiwic2Vzc2lvdj9zdGF0ZSI6ImZkMmU5Yzdh
LTQ2NDUtNDE5OC04NzRjLTk2YmJmMTZiZmRmNCIsImFjciI6IjEiLCJhbGxvd2VkLW9yaWdpbn
MiOlsiaHR0cDovL3B1Y29tZXgt2ViLWFwcC0yLmFwcHMub2

```
{
  "exp": 1652729290,
  "iat": 1652728990,
  "auth_time": 1652728988,
  "jti": "b03bab19-1838-40cd-9bdd-cff92b6694bb",
  "iss": "https://sso-dev-sso-dev.apps.ocp.sefaz.ma.gov.br/auth/realms/sefaz-internos",
  "aud": "account",
  "sub": "d5cd0eaf-05cb-42e2-8727-a62d4c1ac696",
  "typ": "Bearer",
  "azp": "pucomex-public-client-dev",
  "nonce": "a5379915-af34-4535-877f-c5ac2b7e409e",
  "session_state": "fd2e9c7a-4645-4198-874c-96bbf16bdfd4",
  "acr": "1",
  "allowed-origins": [
    "http://pucomex-web-app-2.apps.ocp.sefaz.ma.gov.br",
    "https://pucomex-dev.sefaz.ma.gov.br",
    "http://localhost:4200"
  ],
  "realm_access": {
    "roles": [
      "default-roles-sso-dev",
      "offline_access",
      "uma_authorization"
    ]
  },
  "resource_access": {
    "pucomex-public-client-dev": {
      "roles": [
        "gestor-pucomex"
      ]
    },
    "account": {
      "roles": [
        "manage-account",
        "manage-account-links",
        "view-profile"
      ]
    }
  },
  "scope": "openid profile email",
  "sid": "fd2e9c7a-4645-4198-874c-96bbf16bdfd4",
  "email_verified": false,
  "name": "teste gestor pucomex",
  "preferred_username": "20717107205",
  "given_name": "teste gestor pucomex",
  "email": "testegestorpucomex@sefaz.ma.gov.br"
}
```

Após recepcionado pelo Front-end Angular, é necessário realizar a comunicação com o back-end springboot, no qual a camada front repassa para a camada back o token e este então realiza a validação com o Keycloak.

Com isso, qualquer requisição de serviço da página angular para o back spring, caso necessite um nível a mais de segurança, será feito repassando o token para o back-end e este sendo validado com o Keycloak, e só então o back-end irá executar o seu serviço devido, como se visualiza na figura 2 abaixo.

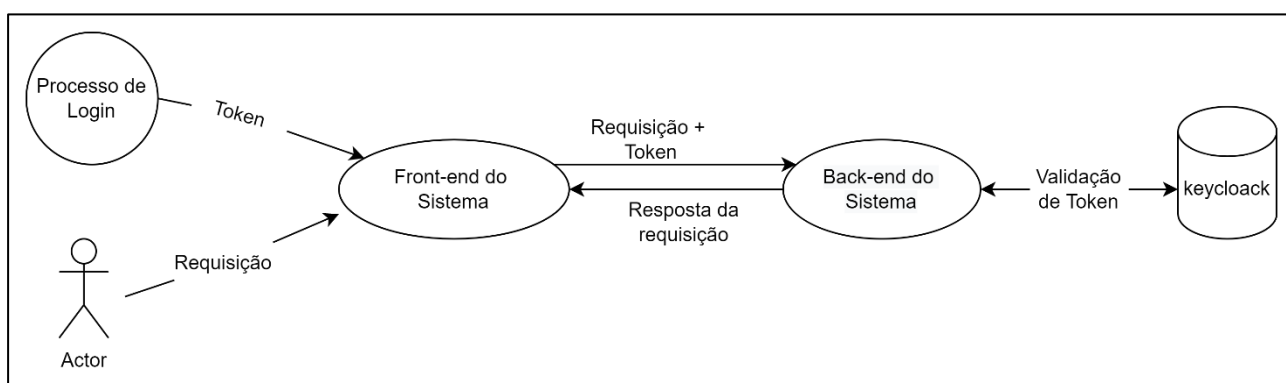


Figura 2-Fluxo de Requisição com token

Tanto o Front-end quanto o back-end (1 ou mais API's) deverão utilizar o mesmo Client, ao qual será um client por sistema ou software.

5. SOLICITAÇÃO DE INTEGRAÇÃO

Para que um sistema possa ter a sua integração realizada ao SSO da SEFAZ-MA, deverá ser feita uma solicitação por email pelo Sevidor da Cotec responsável pelo sistema (Gestor de Sistema, Colider de TI ou Lider de Projeto) ao setor de Segurança/Cotec.

A solicitação irá conter os dados necessários para a criação do Client/Resource, as Roles (se necessário) e os usuários de Acesso no Keycloak. Vale lembrar que esta primeira solicitação é para integração do SSO para desenvolvimento/Homologação. Posteriormente será necessário nova solicitação para produção.

Para a etapa de desenvolvimento, o responsável da COTEC também deve solicitar acesso a VPN com permissão de acesso ao SSO para o Desenvolvedor.

As informações necessárias para a integração são:

CLIENT (cadastro do Sistema):

- Nome do sistema/sigla
- Descrição
- Access Type: PUBLIC/CONFIDENCIAL

(referente ao tipo de acesso ao SSO: Public o login será direcionado ao SSO pelo Angular, Confidencial o login será interno no springboot com api e validação no SSO)

- Valid Redirect URL

(url do sistema que o sso deve redirecionar após autenticação – para desenvolvimento, pode ser localhost)

- Refresh Token: Habilitado/Desabilitado

(permite que seja gerado Tokens de acesso adicionais sem precisar solicitar credenciais novamente sempre que um token expirar.)

- Tipo de usuarios do sistema: interno(servidor)/externo(publico)

ROLE (perfis de acesso):

- Nome do Perfil/sigla
- Descrição

USER (usuário de acesso):

- Nome
- CPF
- Email
- Tipo: consultor/servidor

Para a etapa de desenvolvimento é indicado o cadastro de apenas usuários dos desenvolvedores ou também do Servidor responsável que irão testar a integração assim como as Roles.

Depois de configurado a integração, a COTEC irá fornecer as configurações para os desenvolvedores do sistema configurarem da forma necessária o seu sistema:

- Configurações do REALM (em que o Client se encontra)
- redirect URI
- Client ID
- Role ID's